

Artificial Intelligence for Digital Characters

Exercise 2: Large Language Models

Hand-out Date: 1 April 2026



Goals

- Understanding the fundamentals and methodologies of text embedding and response generation.
- Understanding approaches for adapting Large Language Models (LLMs), such as model fine-tuning and prompt engineering.
- Understanding multimodal data processing and personality infusion basics and techniques.
- Integrating an LLM into a Unity project to enable the avatar to generate a response given the transcription of the user input.

Resources

The lecture slides, exercise slides, and Unity template are accessible via Moodle.

Tasks

1. Multiple Choice Questions

Please mark the correct answer for the multiple-choice questions below.

a) In response generation, the nucleus sampling (Top- p) strategy will randomly pick from the p most likely words.

☐ True ☐ False

b) Prompt engineering is the art of designing effective prompts to guide AI models in generating better responses. More detailed prompts always lead to better AI-generated responses.

☐ True ☐ False

c) Due to the high complexity of LLMs, Parameter-Efficient Fine-Tuning (PEFT) is used to adapt LLMs while significantly reducing the total number of model parameters.

☐ True ☐ False

d) In-context learning refers to training LLMs with prompt-response pairs to improve model performance.

☐ True ☐ False

e) Retrieval-Augmented Generation (RAG) is a technique that enhances language model outputs by incorporating external knowledge retrieval. Instead of relying solely on pre-trained knowledge, RAG dynamically retrieves relevant documents or data from an external source (such as a database or search engine) to generate more accurate responses.

☐ True ☐ False

f) Mean opinion score and A/B testing are both qualitative evaluation methods for chatbots, and both rely on pairwise comparisons between two chatbot versions.

☐ True ☐ False

g) A chatbot with multi-modal inputs and outputs can enhance user experience by adapting its responses based on user state modeling, such as detecting emotions from voice tone or facial expressions.

☐ True ☐ False

h) In a multi-modal chatbot, late fusion always outperforms early fusion because it preserves individual modality representations and prevents information loss.

☐ True ☐ False

i) The five-factor model (openness, conscientiousness, extraversion, agreeableness, and neuroticism) is widely applied to model the personality of humans and chatbots.

☐ True ☐ False

2. Text Representations and Fine-Tuning Techniques

a) Consider two different text representations in natural language processing: Bag of Words (BoW) and Word Embeddings (e.g., Word2Vec, GloVe). Explain how these two methods differ in capturing semantic meaning, and discuss one advantage and one limitation of each approach.

b) What are Parameter-Efficient Fine-Tuning (PEFT) and pruning in deep learning, and how do they differ regarding model optimization for resource-constrained environments? Provide examples of when each technique might be more beneficial.

c) An auto-regressive Large Language Model (LLM) is predicting the next word in the sentence: "**For breakfast, I usually eat a bowl of fresh...**" Which words will be included in the final sampling pool if we apply top-k sampling with $k=3$? What if we apply top-p (nucleus) sampling with $p=0.8$?

Words and their probabilities (after softmax):

Rank	Word	Probability (P)
1	strawberries	0.35
2	fruit	0.25
3	yogurt	0.15
4	cereal	0.10
5	oatmeal	0.07
6	berries	0.05
7	air	0.03

d) Consider the initial prompt given to a language model: "*What is the square root of 144?*"

1. Please improve the prompt using Chain-of-Thought Prompting to encourage step-by-step reasoning.
2. Please improve the prompt using Self-Ask Prompting to break the problem into sub-questions before arriving at an answer.

e) In the Word2Vec algorithm, we perform word embedding by finding the vector representations that maximize the data likelihood. Given the data likelihood $L(\theta)$ for an embeddings model θ ,

$$L(\theta) = \prod_{t=1}^T \prod_{-m \leq j \leq m} P(w_{t+j} | w_t; \theta),$$

we can formulate the negative log-likelihood objective function $J(\theta)$:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log P(w_{t+j} | w_t; \theta).$$

Note that the probability is defined as

$$P(w_o | w_c; \theta) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)},$$

where u_i indicates the vector representation of the corresponding word w_i . Please minimize the objective $J(\theta)$ to maximize the data likelihood $L(\theta)$ by computing the gradient $\nabla J(\theta)$ for one summand with respect to v_c (i.e., compute $\frac{\partial}{\partial v_c} \log P(w_o | w_c; \theta)$).

3. LLM Evaluation and Information Retrieval

a) How do in-context learning and instruction tuning improve the performance of LLMs, and in what scenarios would each method be more effective?

b) Information retrieval (IR) techniques retrieve documents likely to contain relevant information to answer a question. Consider the following query and 3 candidate documents:

- **Query:** *deep learning transformers*
- **Document 1:** *Transformers have revolutionized deep learning by enabling long-range dependencies.*
- **Document 2:** *Deep neural networks are widely used in Chatbots.*
- **Document 3:** *Deep learning models have achieved state-of-the-art results in natural language processing.*

Please compute the tf-idf scores and rank the documents according to their similarity to the query.

c) Perplexity is a common quantitative metric for evaluating LLMs. Given the following two responses:

- **Response 1:** *I like chatbot*
- **Response 2:** *chatbot like I*

Please compute the perplexity of each response according to the probability in Table 1 and explain which response is better.

Table 1: Probability table.

Condition c	Next word w	Probability $P(w c)$
<start>	I	0.5
<start>	chatbot	0.3
I	like	0.6
chatbot	like	0.2
I like	chatbot	0.4
chatbot like	I	0.1

4. Multimodal Chatbots and Personality Design

a) A company wants to develop a customer service chatbot that supports multi-modal interactions, including text, speech, and image inputs. Additionally, the chatbot should adapt its responses based on user state modeling, considering factors like user sentiment, conversation history, and engagement level.

- (i) Explain how multi-modal inputs/outputs can improve the chatbot's effectiveness in customer service.
- (ii) Describe how the chatbot can use user state modeling to personalize its responses. Provide an example scenario where user state modeling changes the chatbot's behavior.

b) In emotion recognition, multi-modal fusion strategies are used to combine different input modalities (e.g., speech, facial expressions, physiological signals) to improve accuracy. Compare early fusion, late fusion, and hybrid fusion in the context of emotion recognition. What are the advantages and challenges of each approach?

c) Imagine you are developing a mental health support chatbot. How would you design its personality traits using the five-factor model for chatbots (artificiality, conscientiousness, vibrancy, civility, and neuroticism) to make it more effective? Please select the appropriate level for each factor (very low, low, moderate, high, and very high) and justify your choices based on user needs and emotional sensitivity.

Project

Project Stages:

1. **Speech Recognition Integration:** Implement speech-to-text capabilities using OpenAI's Whisper to transcribe spoken words into text. Display the transcriptions either in the console or on the screen.
2. **Large Language Model (LLM) Integration:** Connect a large language model, such as Llama or DeepSeek, to enable the character to generate text-based responses.
3. **Speech Synthesis Integration:** Incorporate speech synthesis technologies like FastSpeech 2 to provide a voice for the character.
4. **Character Animation Enhancement:** Enhance the character with animations, including facial expressions using tools like Audio2Face and body animations through platforms like Mixamo.
5. **Designing and Implementing Dialog Tree:** Create a dialog tree in Twine that links user inputs to narrative paths. Implement it in Rasa by defining intents, slots, and stories to ensure smooth transitions.

Stage 2: Integrating a Large Language Model

In this stage, you will enable Vincent to generate text-based responses by integrating an LLM such as Llama or DeepSeek. This involves setting up a connection between Unity and an LLM, processing user input, and displaying the generated responses within Unity.

1. Choose a Cloud-Based LLM or Local Model

- Decide whether to use a cloud-based LLM (e.g., DeepSeek on [Open-Router](#). A tutorial video can be found [here](#)) or a locally hosted model (e.g., [DeepSeek](#) or [Llama](#). A tutorial video can be found [here](#)).
- If using a cloud-based LLM, sign up for an API key from the provider's website. If using a local model, install the necessary dependencies.
- You can decide which route to take (cloud-based LLM or local LLM) according to the following checklist:
 - (a) Computing Power
 - High-performance machine/GPU available? → Consider local hosting.
 - Limited computing resources? → Opt for cloud-based LLM.
 - (b) Latency & Performance Needs

- Need low latency (fast response)? → Local is preferable.
- Tolerable delay? → Cloud-based LLM is fine.
- (c) Data Privacy & Security
 - Handling sensitive/confidential data? → Local hosting is safer.
 - No strict privacy concerns? → Cloud-based LLM is easier.

2. Set Up the Python Flask Server for LLM Communication

- Extend your existing Flask server to include an endpoint that handles text input and generates responses using the LLM.
- Learn how to use the LLM and how to structure chatbot interactions using system, assistant, and user messages and how to maintain conversation history to provide context-aware responses. The [Hugging Face documentation](#), the [Hugging Face NLP Course](#), and the [OpenRouter documentation](#) are good starting points.
- Try to play around with the prompt of the LLM to adapt the personality of Vincent and the characteristics of the conversation. Do not forget to tune the parameters of the LLM (in particular, *max_tokens*, *temperature*, *top-p*, and *top-k*)

3. Configure Unity to Communicate with the LLM Server

- Implement Unity's `UnityWebRequest` to send the transcription from the user's input to the Flask server.
- Ensure the request is structured correctly, including handling JSON formatting.
- Receive and parse the LLM-generated response in Unity.

4. Display the Generated Response in Unity

- Create a UI element (e.g., a text box) to display Vincent's responses.
- Ensure that each response is formatted properly.

5. Test the Integration

- Input text manually or use transcribed speech from Stage 1 to generate responses.
- Verify that the Flask server correctly processes requests and that responses appear in Unity.
- Debug any issues with API requests, response formatting, or UI display.

After completing this stage, Vincent will be able to engage in basic conversations. If you encounter any issues, feel free to post your questions on the Moodle forum.